Universidad Nacional Autónoma de México
Facultad de Ingeniería
Fundamentos de Sistemas Embebidos
Tarea 7: Límites claros
Rafael Ortega de la Paz | 420054085
14/10/26

En el octavo capítulo de Código Limpio, Robert C. Martin parte de un hecho inevitable: casi nunca controlamos todo el software de nuestros sistemas. Martin explica que usamos paquetes de terceros, código abierto o componentes de otros equipos, y que debemos integrarlos con nuestro código sin que eso ensucie la aplicación. Robert señala además que existe una tensión natural entre ambas partes: el proveedor diseña una interfaz amplia para atraer al mayor número de usuarios, mientras que nosotros sólo necesitamos una parte concreta de ella.

Para ilustrarlo, Martin recurre a java.util.Map. Según el autor, su interfaz es tan amplia que cualquiera que reciba el mapa puede borrarlo con clear() o guardar en él objetos de cualquier tipo. Robert advierte también que, si Map se pasa por todo el sistema, un cambio en su interfaz, como el que ocurrió al añadir genéricos en Java 5, obliga a corregir muchos puntos del código. La solución que propone Martin es encapsularlo dentro de una clase propia, como Sensors, que sólo ofrezca lo que la aplicación necesita y mantenga oculta la interfaz de límite.

Antes de integrar código de terceros, Robert recomienda escribir pruebas de aprendizaje: experimentos controlados que invocan la API tal como la usaríamos. Martin narra cómo, con log4j, descubrió a base de pruebas que necesitaba un ConsoleAppender y un PatternLayout sólo para imprimir un simple "hello". Para Robert, estas pruebas no cuestan nada, pues de todas formas había que aprender la API, y cuando sale una nueva versión del paquete basta con volver a ejecutarlas para saber si algo cambió.

Finalmente, Martin habla de los límites con código que todavía no existe. Robert cuenta que, en un sistema de radio, su equipo definió la interfaz Transmitter que deseaba tener y, cuando la API real estuvo lista, la conectó mediante un TransmitterAdapter. Martin concluye que es mejor depender de algo que controlamos que de algo que no controlamos, y que deben reducirse al mínimo los puntos del código que conocen los detalles de terceros.